TENDERGUARD

Privacy-Preserving Multi-Agent RAG Platform for Automated Tender and Compliance Analysis for SMEs

Daniyar Kairatov

Rassul Akhmetov

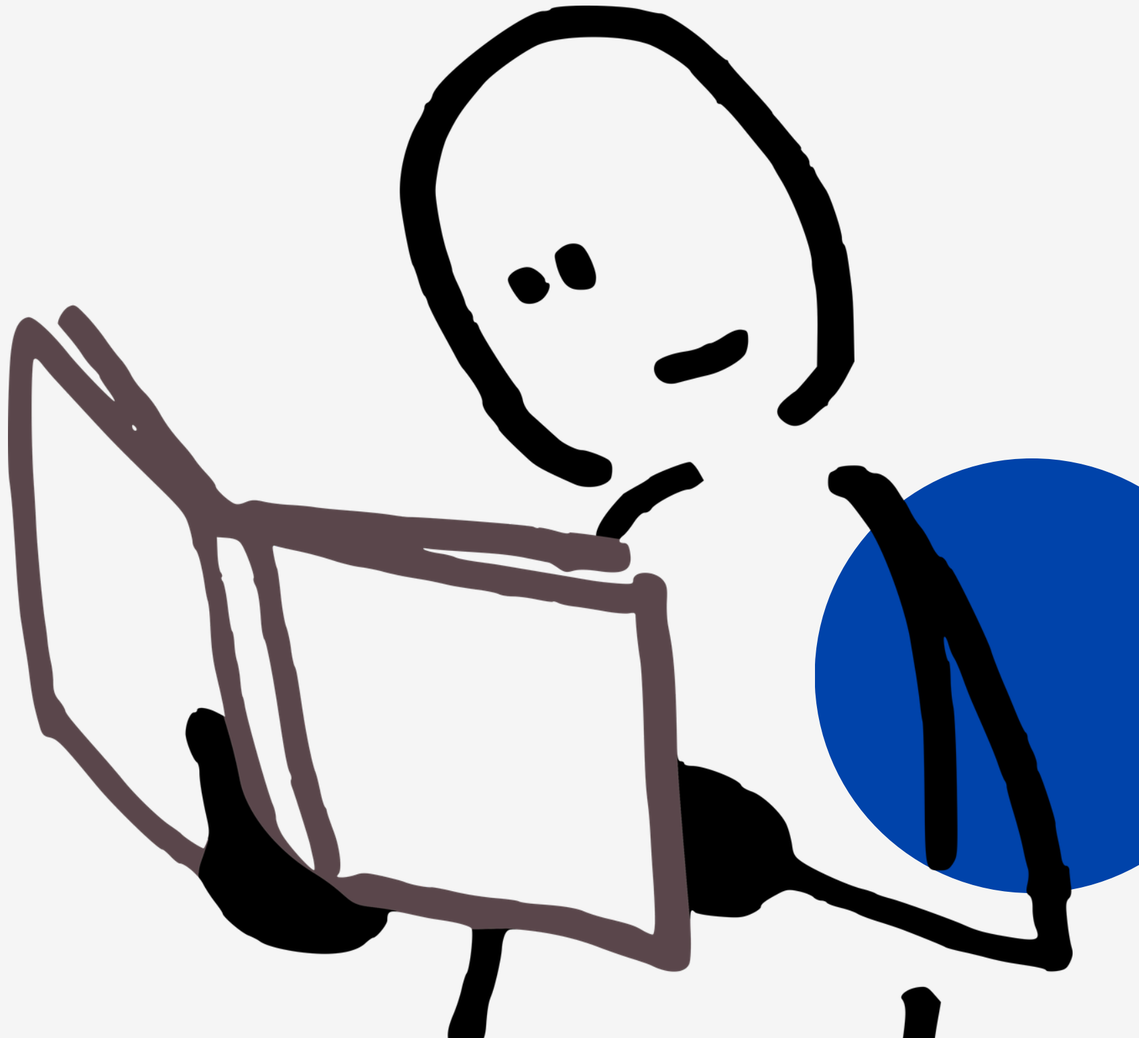
Ramazan Talgatov



Problem, Users and Aim

SMEs must prove every tender requirement with their own company documents

Conditions are spread across specifications, attachments and amendments. Specialists check each one against company documents by hand.
Users: SME tender specialists and bid managers in Kazakhstan, RU/KK documents.
Aim: extract requirements, match them with company evidence, and show citations and uncertainty a human makes the final decision.



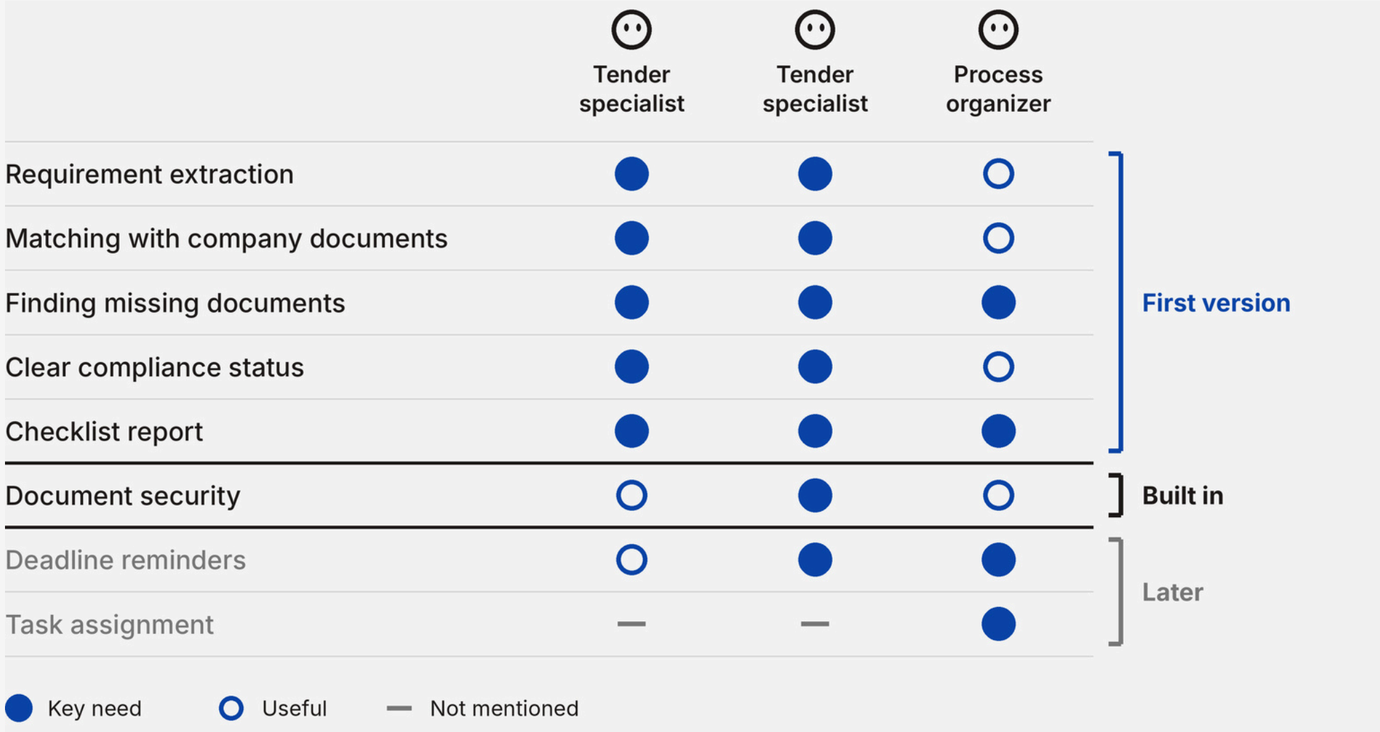
EVIDENCE OF RELEVANCE: THREE EXPERT INTERVIEWS

We interviewed two tender specialists and one tender process organizer. Today they read tender files manually, make checklists by hand and search for certificates, licences and contracts in different folders. Missing or outdated documents are often found too late.

Specialists want to see each requirement with its source, the matching company document and a status. When evidence is not enough, the system should say “needs review”.

What we take into the first version: extraction, document matching, clear statuses and a checklist report. Deadlines and task assignment come later.

Requirements spread across files
Missing documents found late
Outdated evidence goes unnoticed
AI results must be explained





LITERATURE REVIEW

12 scientific publications
3 research areas
3 team contributors
1 shared synthesis

Agents and evaluation — Rassul. ReAct, MAST, LegalBench and RAGAs show that multi-agent systems often fail at handoffs and that legal reasoning is hard. So we validate every stage output and keep document checks separate from legal advice.

RAG security — Ramazan. Four studies on privacy leakage, document poisoning and prompt injection show that retrieved text can leak data or manipulate a model. So every model call passes a gateway, and documents are treated as data, not instructions.

Retrieval and citations — Daniyar. BGE-M3, Self-RAG, ALCE and Lost in the Middle show that multilingual search, citation support and evidence order all affect answer quality. So we compare retrieval methods and check that each citation really supports the finding.

Our synthesis. Together these papers shaped our method: retrieval, then extraction, assessment and reporting, compared with a single-prompt baseline. Their results come from other data, so we will measure everything on Russian and Kazakh tenders.

EXISTING SYSTEMS

5 platforms from Kazakhstan, Russia and Uzbekistan

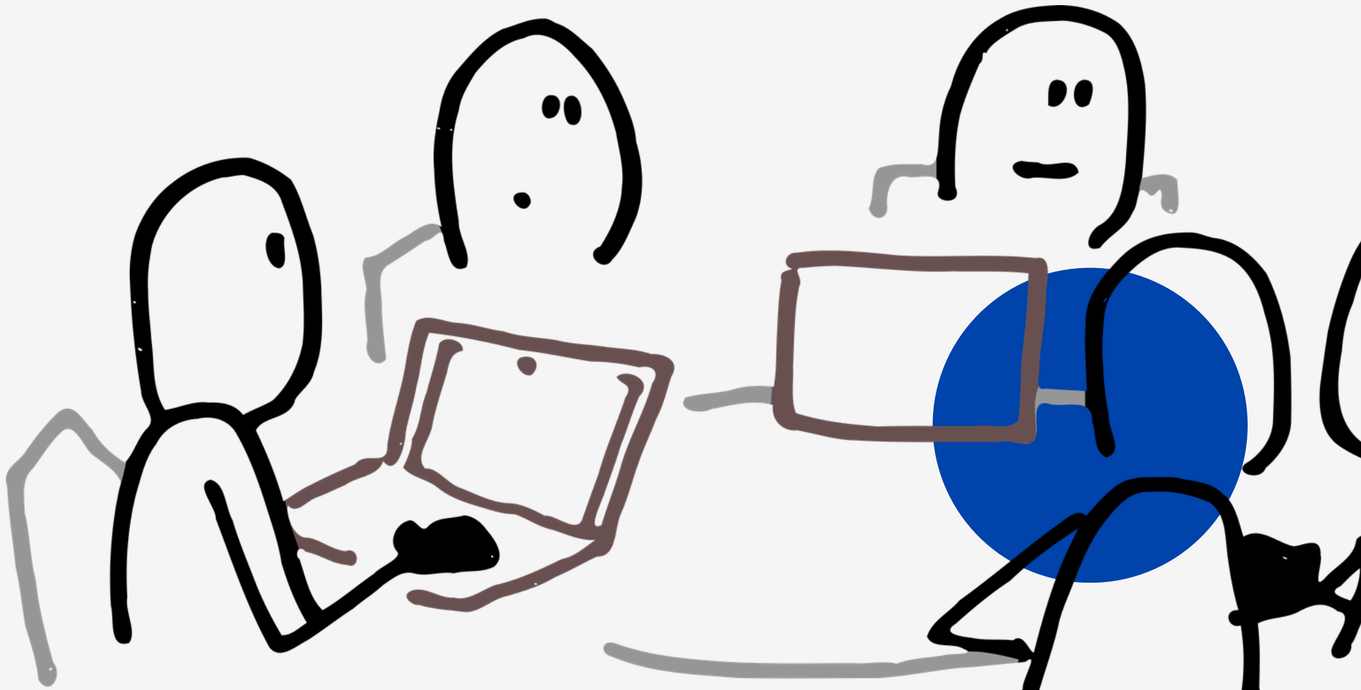
10 criteria, applied the same way to each

01

Tenderbot, Tenderplan, Kontur.Zakupki, BiCoTender and UZEX eTender help companies find tenders, set alerts and search documents. Kontur and BiCoTender also offer AI summaries of conditions and risks. So a tender summary or a risk report alone is not new, and we do not present it as our contribution.

02

In the materials we reviewed, none of the five systems shows that it matches tender requirements against the company's own documents with a clear status: met, not met or unknown. TenderGuard focuses on exactly this, with citations to both documents, saved review history and control over where company data goes.



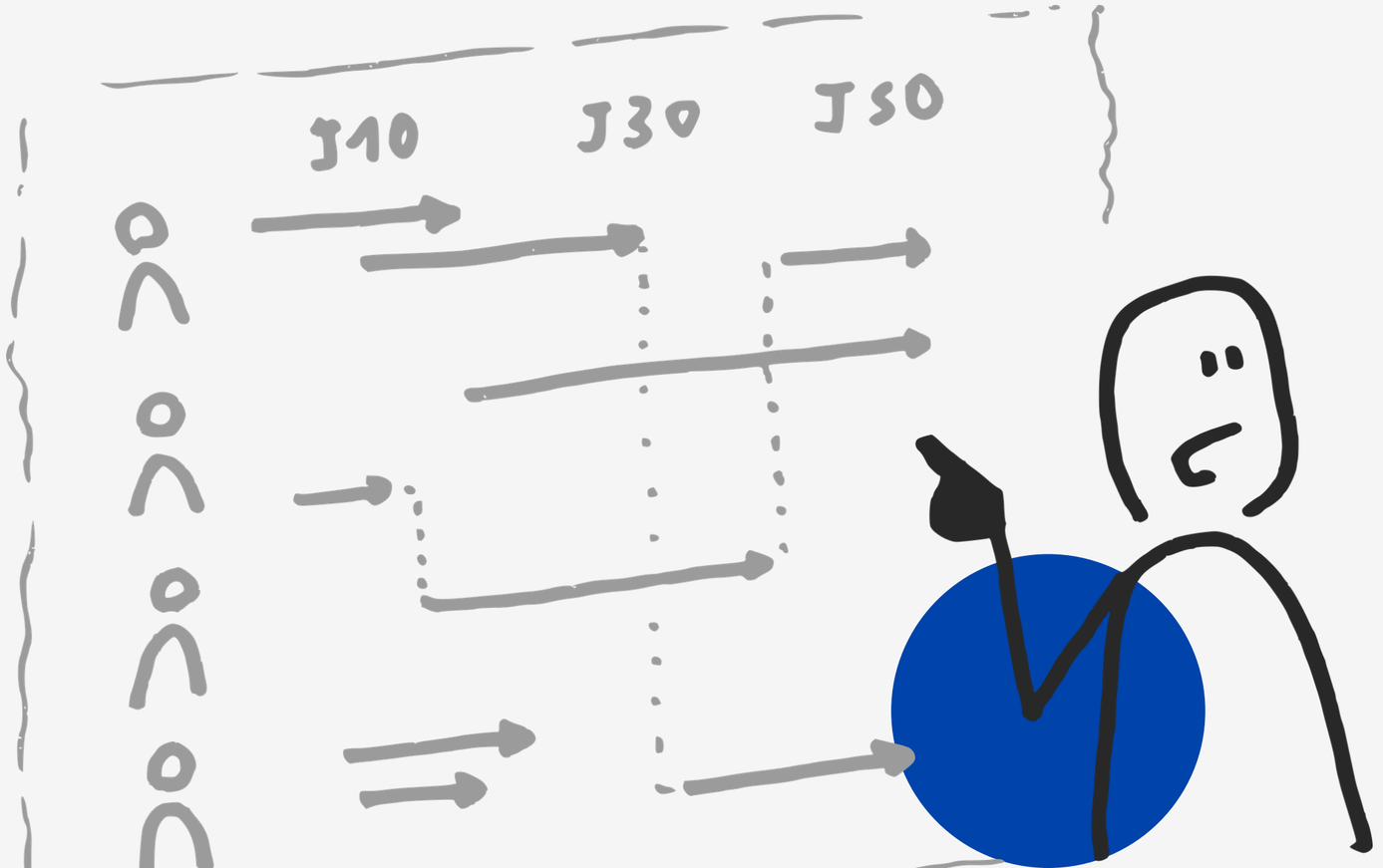
ML STACK

We compared two options for each decision:

- Embeddings: BGE-M3
- Search: pgvector & keyword search
- Pipeline: 3 stages
- Model: GPT-5.4 Mini

We chose BGE-M3 over E5-base because it reads up to 8192 tokens and supports Kazakh better; E5-base stops at 512 tokens. We chose GPT-5.4 Mini over a local Qwen3-4B because it supports structured output and needs no GPU. Qwen3-4B stays as an option for private data.

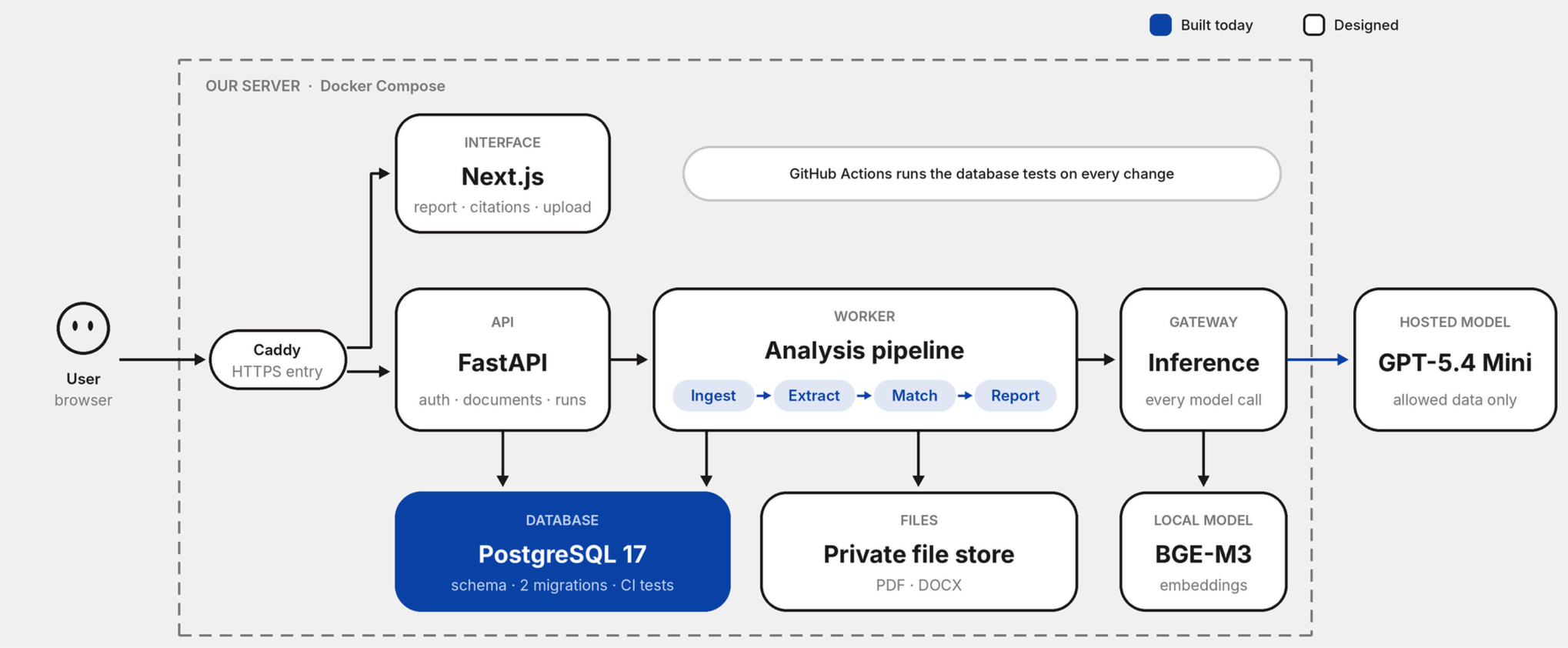
We chose a staged pipeline over a single prompt: one stage extracts requirements, one checks them against company documents, one writes the report. Each step can be checked and explained. We keep vectors in PostgreSQL with pgvector instead of a separate vector database, and use plain Python with Pydantic instead of a heavy agent framework. The single prompt stays as a baseline to prove the stages help.



BACKEND

PostgreSQL schema, 2 migrations and automated tests in CI.

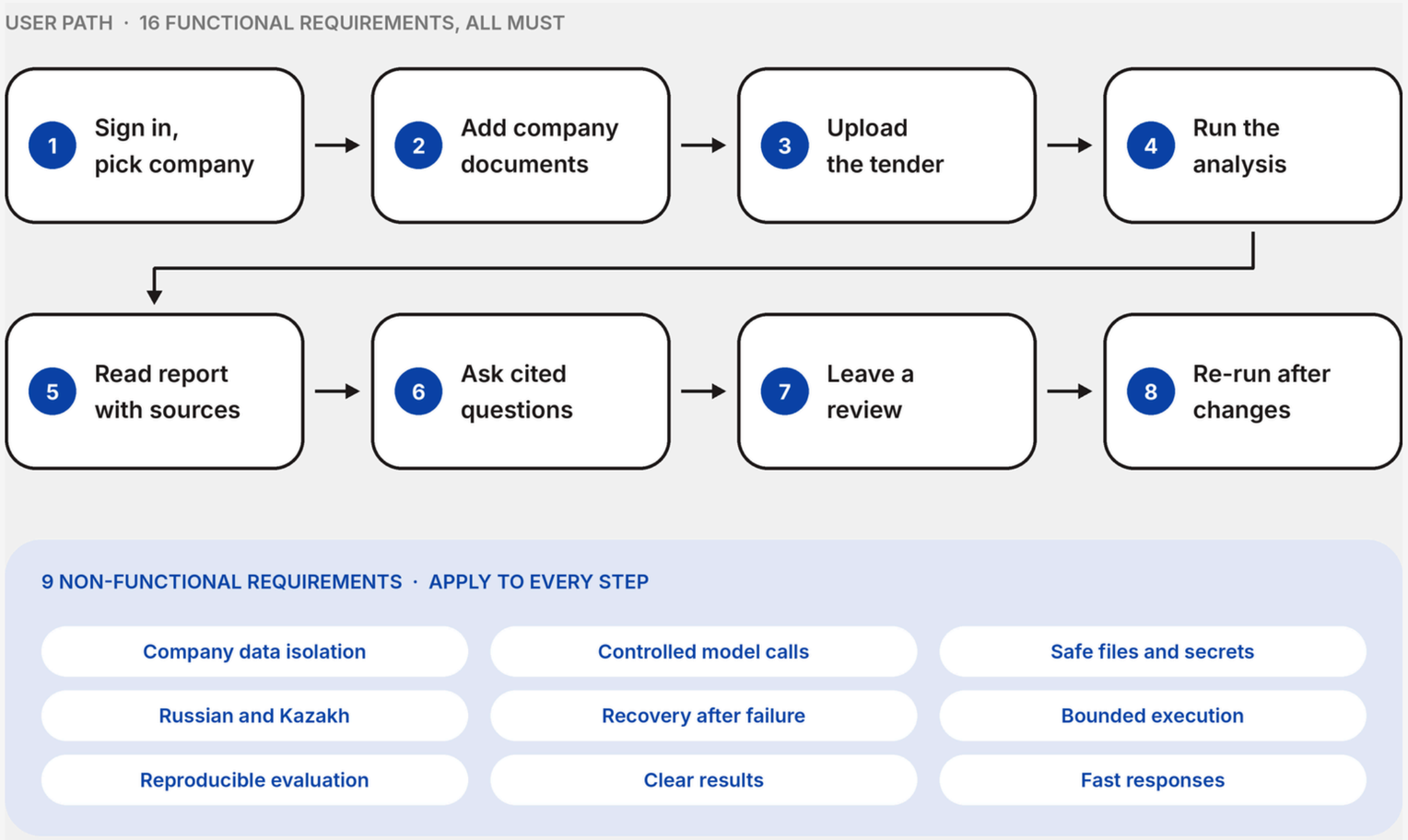
Everything runs on one server with Docker Compose. Only the gateway can reach an external model.



One language with the ML code, one database for documents, results and vectors.

WHAT THE SYSTEM MUST DO

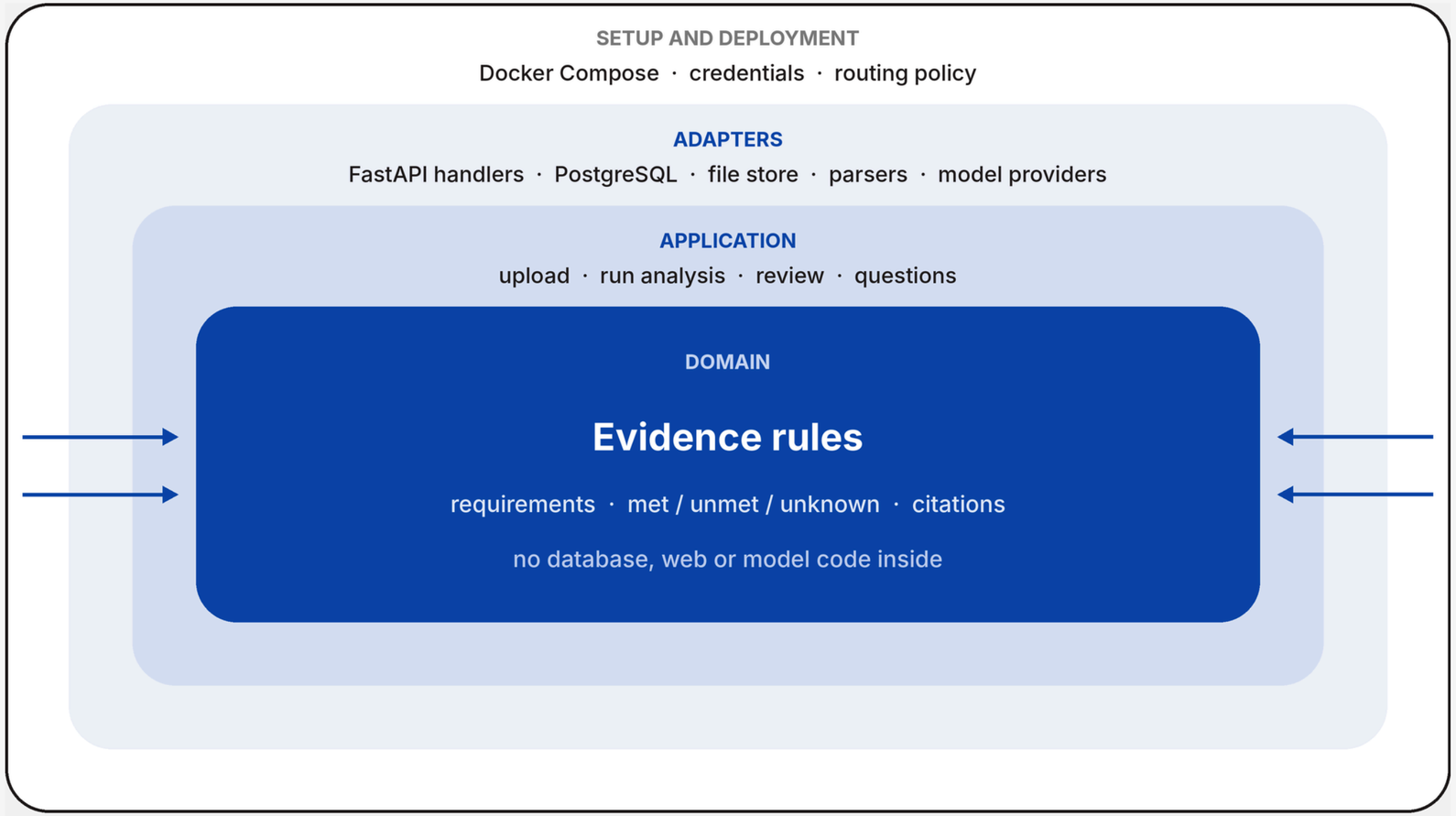
- 16 functional requirements, all Must
- 9 non-functional: security, RU/KK, recovery
- Every requirement has an acceptance test
- Later: OCR, report export, local model



ARCHITECTURE

HOW IT | FITS | TOGETHER

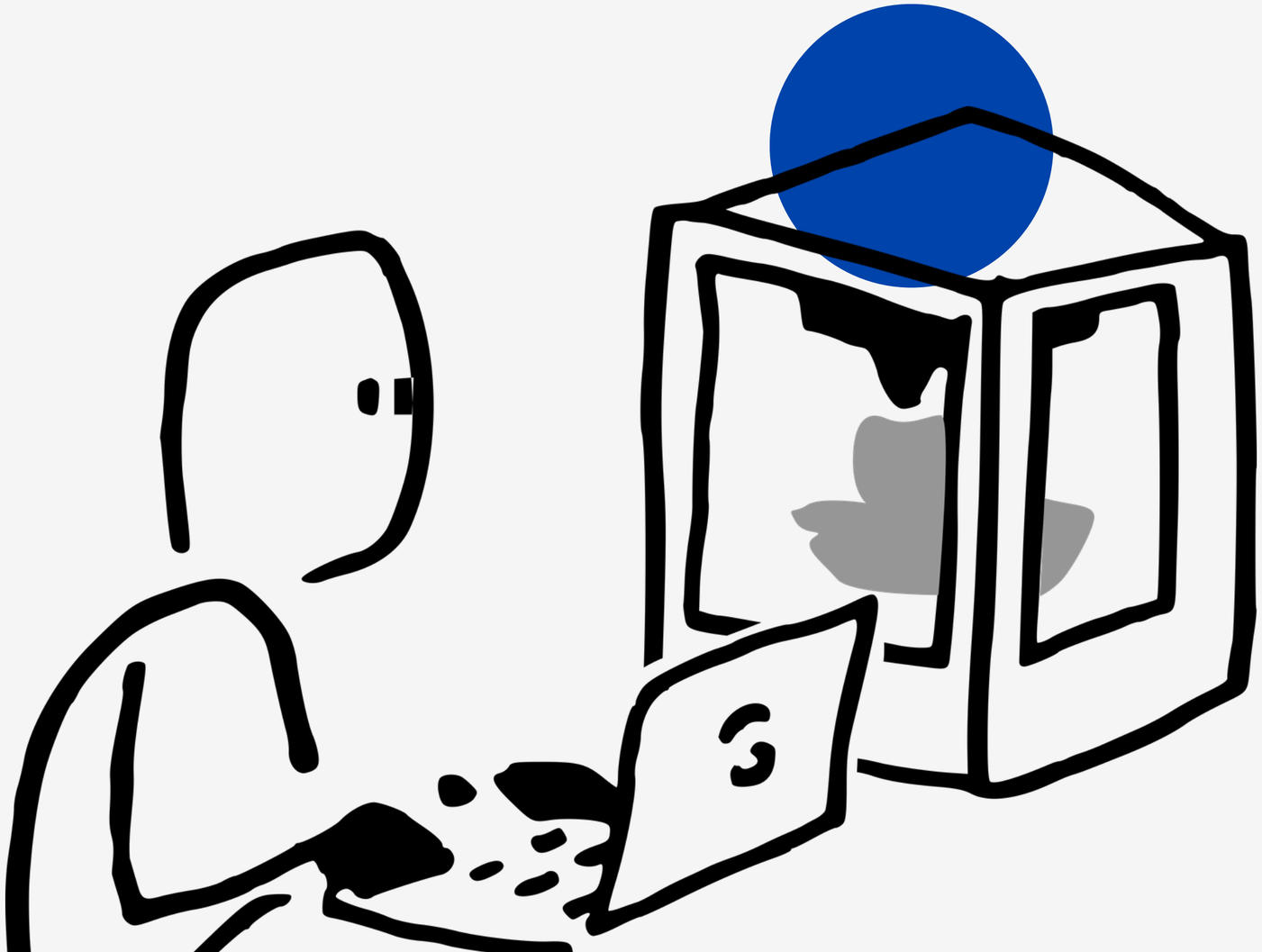
- Built on the supervisor's architecture
- Added: company documents, durable worker, Inference Gateway
- One modular backend, not microservices
- 6 diagrams in the repo with editable sources



Dependencies point inward: we can change a model or a database without touching the evidence rules

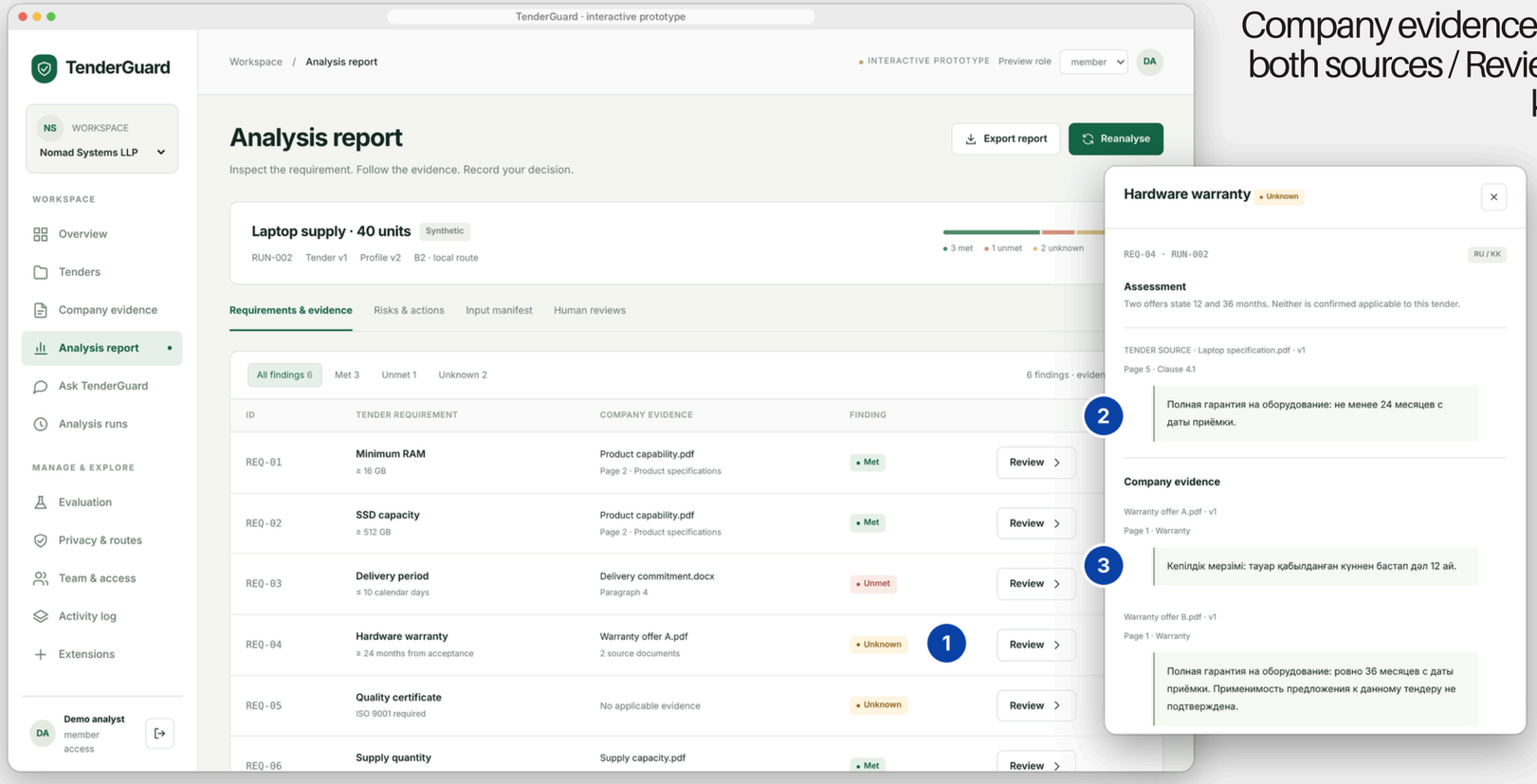
PRIVACY BOUNDARY

Each company sees only its own documents and results. Every model call goes through one gateway, which checks who is asking, which company the data belongs to and whether the route is allowed. If no route is allowed, the analysis stops and nothing is sent outside. A local model alone does not make data private, so we rely on these checks. Today this is designed, and tests with two test companies will prove it.



UX PROTOTYPE

Company evidence - tender - report / Every finding opens both sources / Reviews never overwrite results / New run keeps the old one



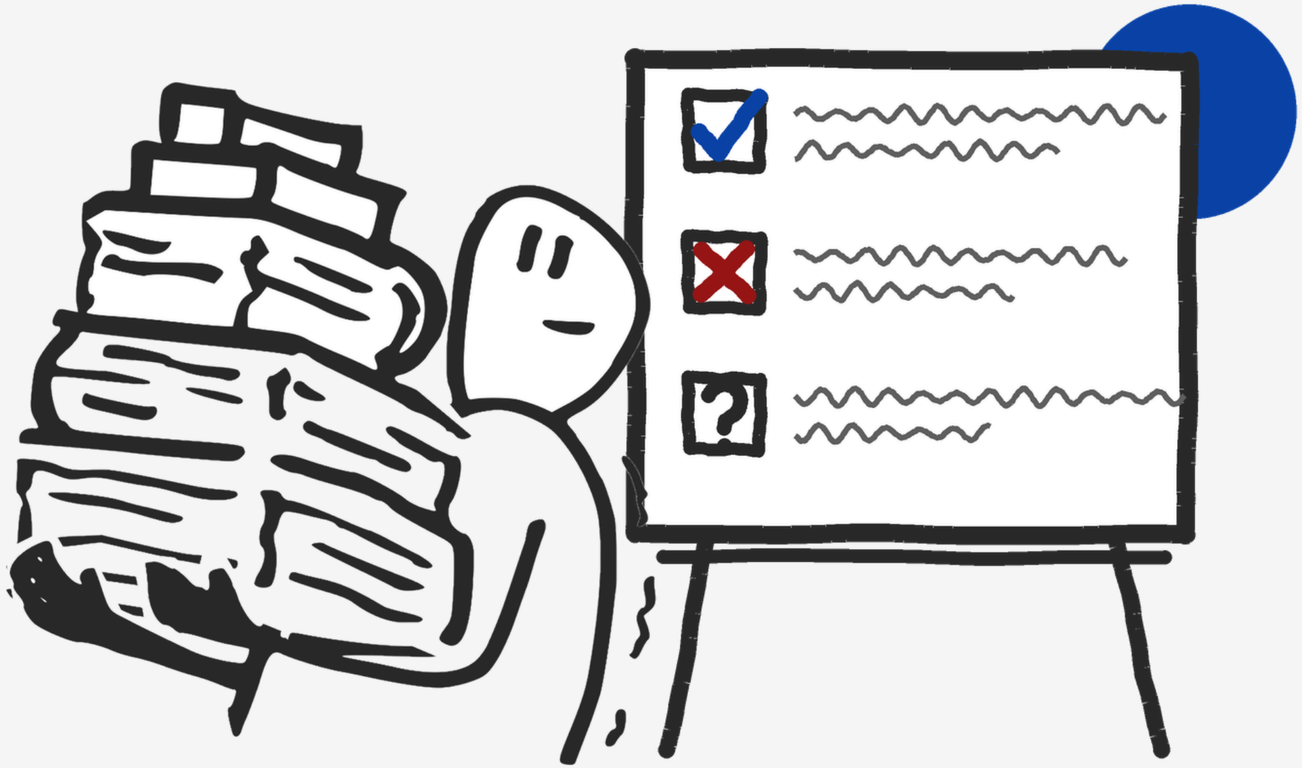
1 Status for every requirement

2 Tender quote next to company evidence

3 Russian and Kazakh documents

WHAT A FINDING LOOKS LIKE

Every requirement gets one of three statuses. Met means the company’s documents fully support it, unmet means they show a violation, and unknown means the evidence is missing, unclear or conflicting. Missing evidence is never unmet. For example, the tender asks for a 24-month warranty, while offer A gives 12 months and offer B gives 36. The tender does not say which offer applies, so the answer is unknown, and both sources stay visible for the user to check. A timeout or a blocked model is not a status, it is a processing error.



EVALUATION PLAN

- RQ1 retrieval · RQ2 staged vs single prompt · RQ3 privacy controls · RQ4 user understanding
- Provisional targets: complete-evidence success@5 ≥ 0.85 · status macro-F1 ≥ 0.80 · supported coverage ≥0.80
- Corpus plan: 30 tender families (18 dev / 12 test) · RU and KK reported separately

Variant	**Method**	**Measured on**
B0	PostgreSQL lexical search	Retrieval only: Recall@5/@10
B1	Same retrieval + one generation call	Statuses, citations, false passes, cost
B2	Same evidence + typed stages	Same, plus every extra call

PROJECT MANAGEMENT AND RISKS

RQ1 Does search find the right evidence?
RQ2 Do stages beat a single prompt?
RQ3 Do privacy controls hold?
RQ4 Can users understand the report?

THREE VARIANTS · SAME DOCUMENTS, SAME MODEL

B0	Keyword search →	Ranked passages			Did search find the evidence?
B1	Same search →	One model call →	Report		Statuses, citations, cost
B2	Same search →	Extract →	Assess →	Report	Same, plus every extra call

PLANNED TARGETS · NOT RESULTS YET

0.85
find the full evidence
in the top 5 results

0.80
correct status
met · unmet · unknown

0.80
requirements found
with evidence

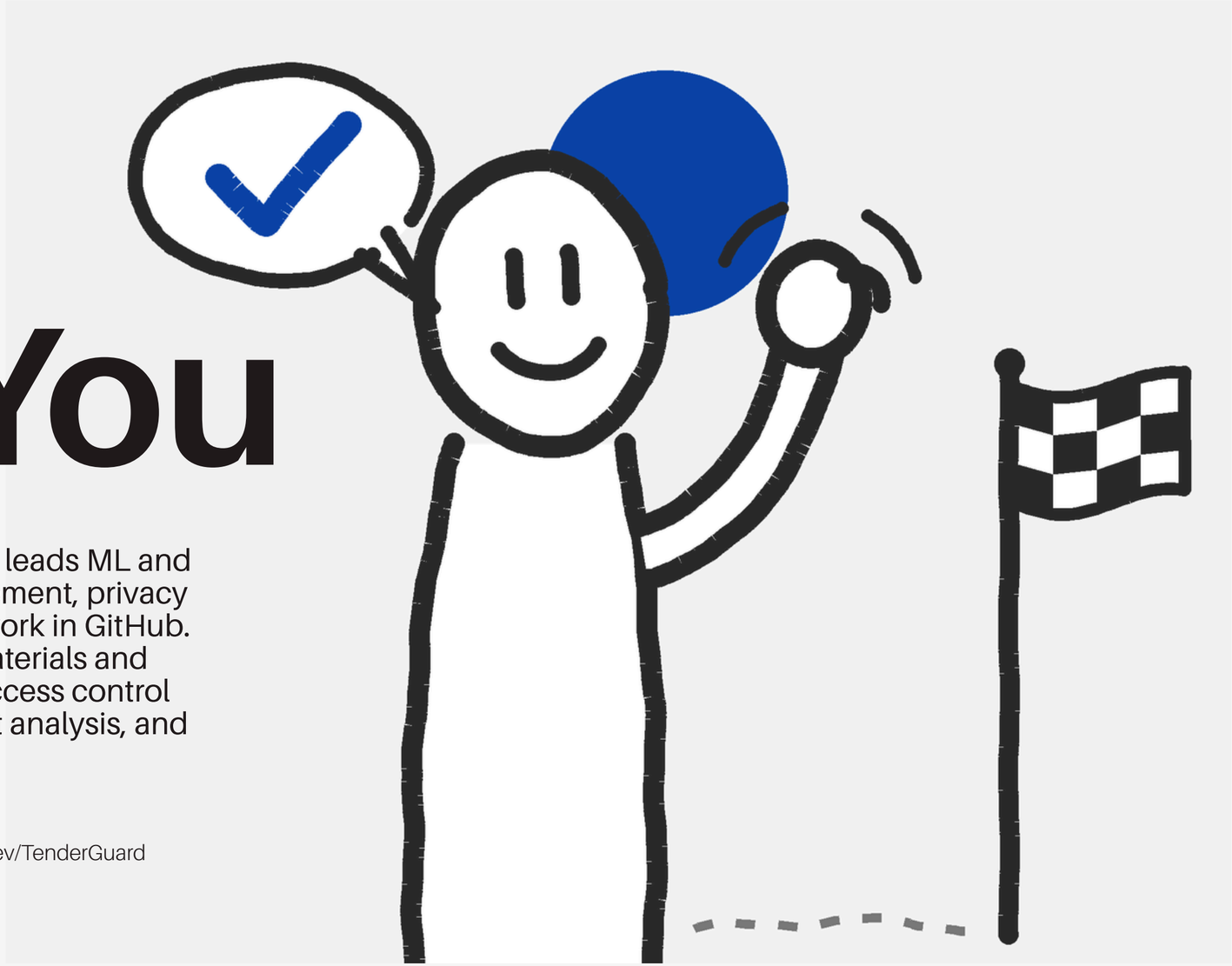
Dataset plan

30 tender families

18 develop | 12 test

Russian and Kazakh scored separately
Related documents never split

Thank You



Rassul leads the backend and analysis, Daniyar leads ML and evaluation, and Ramazan leads project management, privacy and UX. We track tasks in Notion and keep all work in GitHub. Our supervisor guided how we structure the materials and gave us the base architecture. Next, we build access control and document upload, then search and the first analysis, and finally the full evaluation.

github https://github.com/Tortuev/TenderGuard